

Chapter 3: Efficient Tabular Data

Preparation For AI: From Raw Data To Reliable Models

Alexander D'hoore

Efficient Tabular Data

Chapter 2 made object storage the shared place.

Chapter 3 asks:

What file shape makes tabular data fast to reuse?

The answer for this course is **Parquet plus query engines.**



The Problem With Tables In A Lake

The lake can store tabular data as CSV.

But repeated analytics need more:

- stronger types
- less parsing
- fewer bytes read
- reusable outputs

Chapter 3 turns stored files into **efficient reusable datasets**.



The Core Pattern

Parquet is the file format.

Query engines are the workers.

Object storage remains the **source of truth**.



The Platform Pattern

The storage foundation stays stable:

```
shared object storage  
-> multiple compute engines  
-> reusable datasets
```

The dataset should not belong to only one tool.

If the data is only usable by one script, it is not really platform data yet.



CSV: Useful, But Weak For Reuse

CSV is a good exchange format:

- easy to inspect
- easy to produce
- supported almost everywhere

But repeated analytics usually forces tools to:

- parse text
- infer or apply types
- scan rows
- read unused columns
- handle malformed values late

Why Parquet

Prepared tabular data should be:

- typed
- compressed
- splittable
- readable by many engines
- efficient for selected columns
- inspectable through metadata

This is where **Parquet** enters.



Row-Oriented Versus Columnar

CSV thinks in rows.

Parquet stores values from the **same column together**.

Analytical queries often need only a few columns.

CSV / row-oriented

row 1: A B C
row 2: A B C
row 3: A B C
row 4: A B C

Read whole rows

Parquet / columnar

A	B	C
A	B	C
A	B	C
A	B	C

Read needed columns

Parquet Has Useful Metadata

A Parquet file contains:

- schema
- row groups
- column chunks
- statistics
- compression metadata

Query engines can plan before reading all the data.

That metadata is what makes Parquet work well in a data lake.



Parquet Is Not A Database

Parquet stores data efficiently.

It does not answer queries by itself.

You still need a query engine to:

- read metadata
- apply filters
- aggregate rows
- join tables
- write outputs

This is the core partnership:

Parquet files + query engine = analytical dataset

Layout And Partitioning Matter

A dataset in object storage is usually a prefix, not one file.

```
silver/chapter03/coverttype/  
  part-000000.parquet  
  part-000001.parquet
```

Partitions add meaningful prefixes:

```
silver/chapter03/coverttype/  
  wilderness_area=0/...  
  wilderness_area=1/...  
  wilderness_area=2/...
```

Good partitioning matches common filters.

Bad partitioning creates too many tiny files and expensive listings.

The Question Is Compute Placement

Do not ask:

Which tool sounds most impressive?

Ask:

Where should this computation run?

Local tools are often the best first answer.



Polars

Polars is a fast dataframe query engine for Python.

In this chapter, it teaches:

- scan files lazily
- cast columns deliberately
- write partitioned Parquet
- push filters toward the scan



DuckDB

DuckDB is local SQL over analytical files.

No database server is required.

It can read Parquet directly and write Parquet outputs.

SQL makes tabular logic easy to discuss with other people.



Polars And DuckDB Together

They are different interfaces over the same idea:

Polars: Python dataframe queries

DuckDB: SQL queries

The important thing is not which one wins.

The important thing is that both can reuse the same Parquet dataset.



When Local Stops Being Enough

Spark becomes useful when:

- work must run on shared infrastructure
- data is naturally parallel
- many teams depend on the output
- the job will later be scheduled

For tiny data, Spark overhead is real.



Spark On Kubernetes

In the workshop, Kubernetes starts temporary Spark pods.

SparkApplication

- > driver pod
- > executor pod
- > Parquet output in object storage

The durable result is what Spark writes back to object storage.



Lab Orientation

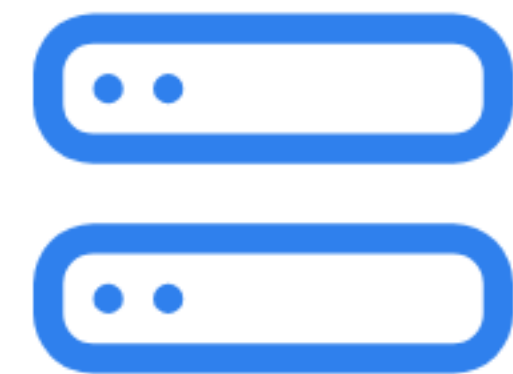
The lab reuses Forest Cover Type.

This time, the modelling problem is not the focus.

The data-platform goal is:

Bronze CSV -> Silver Parquet -> Gold metrics

The same Silver dataset is read by Polars, DuckDB, and Spark.



One Important Distinction

Storage layout and analytical questions are different choices.

In the lab:

- partition column: where bytes live
- group column: what comparison we want

Do not confuse physical layout with business meaning.



Efficient Tabular Data

Efficient tabular data is a lakehouse bridge.

Remember:

- CSV is exchange-friendly, not analytics-friendly
- Parquet is typed, compressed, and columnar
- layout affects reads
- query engines do the work
- object storage stays the source of truth
- good storage lets compute engines change

Next: add table state on top of Parquet files.